Vivekanand Education Society's Institute of Technology

Department of Computer Engineering

Group No.: 03
Date : 11/08/2025

Project Synopsis (2025-26) - Sem VII

AI-Powered Smart City Lighting Systems

Dr. Mrs. Nupur Giri
HoD, Computer Engineering

Ronak Ajwani	Abdul Hannan Chougle	Himanshu Menghrajani	Shreya Chhatwani
V.E.S.I.T	V.E.S.I.T	V.E.S.I.T	V.E.S.I.T
2022.ronak.ajwani@ves.ac.in	2022.abdul.chougle@ves.ac.in	2022.himanshu.menghrajani@ves.ac.in	2022.shreya.chhatwani@ves.ac.in

_____

Dr. Mrs. Nupur Giri
Project Mentor

Abstract

Smart city lighting systems increasingly rely on networked control and cloud services to deliver energy-efficient, adaptive illumination—but that same connectivity broadens the cyber-attack surface. This project develops an agentic AI security layer that operates on network telemetry, API access logs, authentication records, and device performance metrics to detect and mitigate Denial-of-Service (DoS), Man-in-the-Middle (MitM), AI model-poisoning, and telemetry-spoofing threats. The core design is a modular pipeline of lightweight edge and central agents (ingestion → classification → RAG-assisted reasoning → planning → human-in-the-loop enforcement), enabling low-latency mitigation, explainable incident reports, and auditable rollback actions. We will implement the DoS detection–mitigation baseline in a Dockerized testbed and extend the pipeline for integrity threats, evaluating performance using simulated dataset, and synthetic packet traces. Success will be measured by precision/recall/F1 per threat class, false-alarm rate, detection latency, threat-isolation rate, and model robustness under poisoning. Deliverables include the implemented pipeline, testbed scripts, a reproducible codebase, and a comprehensive evaluation report.

1. Introduction

Modern smart-city lighting platforms are cyber-physical systems that increasingly rely on network-level telemetry and distributed control software instead of local physical sensors. Typical deployments include embedded lighting controllers (often grouped into logical zones), gateway devices that aggregate telemetry and enforce local policies, and centralized management applications for scheduling, firmware updates, and analytics; communication may traverse local IP networks, private LTE/5G slices, or cloud APIs. This network-centric design delivers operational benefits (scalability, remote diagnostics, energy optimisation) but also expands the attack surface at the network, application, update, and machine-learning layers [1][8]. Work on smart-lighting and IoT deployments has shown practical exploitation paths—for example, MQTT/ZigBee attack vectors that can be used to compromise lighting control flows—highlighting the real-world risk to these systems [1].

Security studies across IoT and critical-infrastructure domains document several high-impact threat classes relevant to lighting platforms. Volumetric and application-layer Denial-of-Service (DoS) attacks can overwhelm gateways or cloud endpoints and interrupt control and monitoring functions [3]. Man-in-the-Middle (MitM) attacks are able to intercept or modify commands and telemetry, breaking trust assumptions between controllers and management systems [4]. Machine-learning components used for anomaly detection are vulnerable to poisoning and adversarial manipulations, which can steadily degrade detection performance or produce targeted misclassifications [6][5]. Finally, telemetry or heartbeat spoofing—replay or fabrication of device logs—can mask ongoing incidents or generate false alarms that burden operators [7]. National guidance for secure smart-city deployments stresses the need for coordinated detection and response across layers rather than siloed countermeasures [8].

Given these interlocking threats and practical constraints (heterogeneous firmware, constrained edge compute, and requirements for low latency and high availability), we propose an integrated, agentic security layer that coordinates detection, contextual reasoning, mitigation planning, and auditable reporting. The design uses retrieval-augmented techniques to surface past incidents and playbooks during decisioning, couples edge and cloud processing to meet latency and scale requirements, and incorporates human-in-the-loop review for high-impact actions—building on prior work in RAG and federated/anomaly detection approaches for IoT systems [10][2].

2. Problem Statement

Even without visual or motion sensors, smart lighting deployments remain vulnerable to a range of cybersecurity threats that target the network, application, and machine learning layers of the system.

- **DoS/DDoS:** High-volume or carefully crafted application-layer requests can overwhelm gateways or API endpoints, causing legitimate control commands to be delayed or dropped. In dense urban deployments, where thousands of nodes may depend on a small number of gateway devices, such attacks can cause widespread outages and compromise safety-critical lighting schedules.
- **MitM & False-Data Injection:** Attackers positioned in the communication path can intercept, alter, or replay commands and status messages. Examples include forcing lights into unsafe states (max brightness during daytime, complete blackout in public areas) or injecting forged firmware update triggers. Without strong authentication and encryption, these attacks can remain undetected for extended periods.
- **AI Model Poisoning:** Many modern lighting systems rely on ML-based anomaly detection. Adversaries can corrupt these models either at training time (by introducing malicious data into the corpus) or during online learning (via crafted telemetry). This can cause the system to misclassify threats as benign, disable automated responses, or even take unsafe actions.
- **Sensor/Input Spoofing:** In the absence of physical sensors, attackers can fabricate device telemetry—such as heartbeat messages, health metrics, or operational logs—or replay historical data. This misleads decision-making agents into ignoring real threats or triggering unnecessary interventions, leading to both operational inefficiency and security blind spots.

These threats are amplified by systemic challenges: the heterogeneity of device firmware and communication protocols, the constrained computational capacity of edge devices (limiting complex real-time analysis), and the operational need for high availability and low latency in public infrastructure. While defensive measures exist at various layers—such as network firewalls, TLS encryption, or API rate limits—they are often siloed and lack coordinated, context-aware workflows that span the full detection → analysis → mitigation → audit cycle.

Addressing this requires a unified, agent-based security framework capable of integrating multiple detection techniques, correlating events across different layers, and executing safe, reversible mitigation strategies without disrupting the primary lighting functions.

3. Proposed Solution

We will develop an agentic cybersecurity framework operating solely on network telemetry, system logs, and API records—removing dependence on physical sensors. The design centres on a unified data-processing pipeline, adaptable for multiple attack types:

- **DoS baseline:** High-frequency request detection and throttling.
- **MitM extension:** TLS/DTLS certificate validation and session monitoring.
- **Model poisoning protection:** Update verification and sandbox testing.
- **Sensor/Input Spoofing prevention:** Timestamp/signature checks and anomaly scoring.

The architecture emphasises:

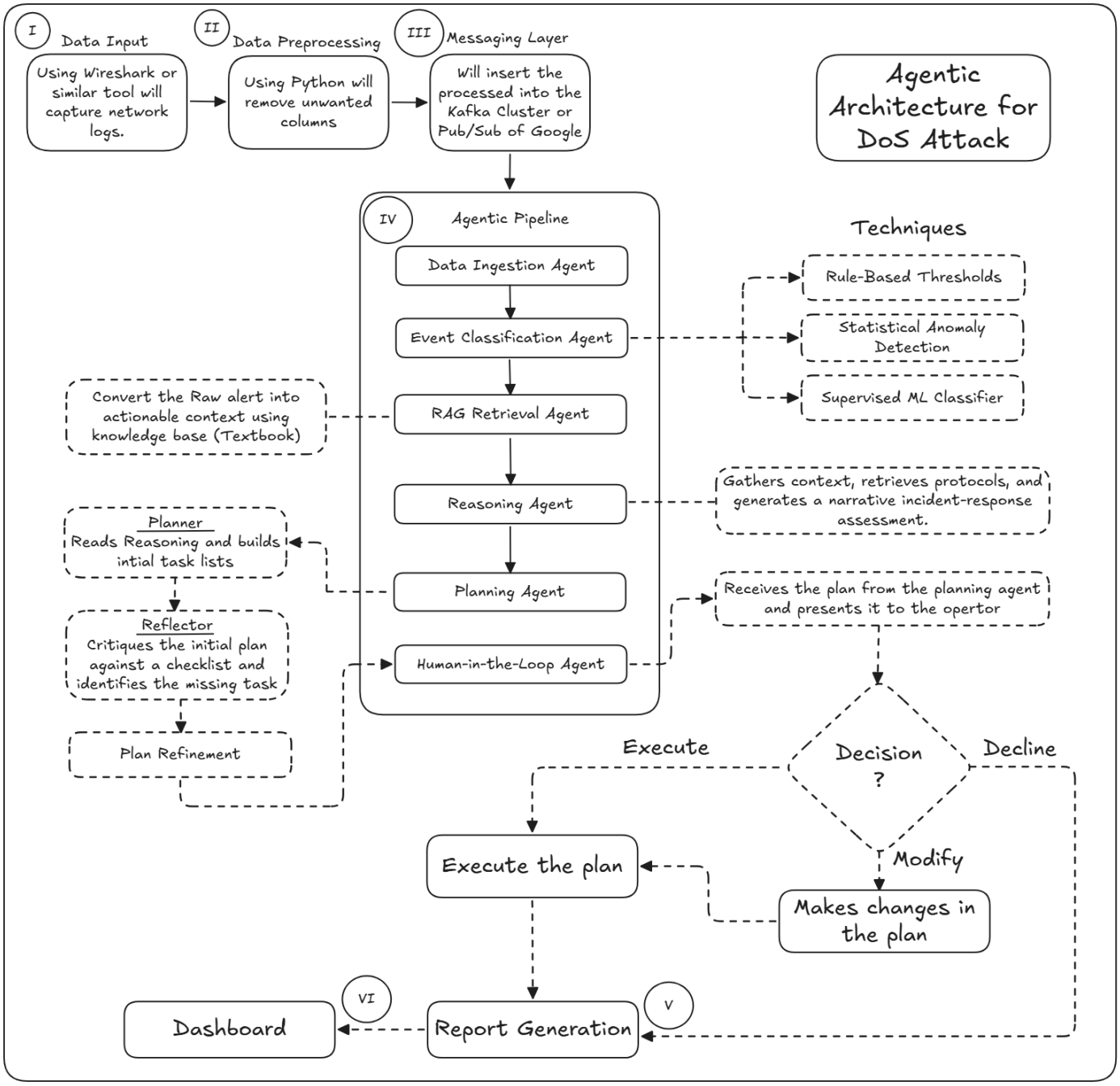
- **Modularity:** Independent updates to each functional block.
- **Scalability:** Capable of handling city-scale infrastructure.
- **Explainability:** RAG-based retrieval of past incidents and technical playbooks.
- **Safety:** Human oversight for high-impact mitigation steps.

Attack Type	Key Vulnerability	Agentic Mitigation
DoS/DDoS	API endpoints, agent interfaces	Throttle via DoS agent, anomaly detection, rate-limiting
MitM	Unsecured communication, weak authentication	TLS/DTLS, session-monitoring agent, end-to-end Encryption
Model Poisoning	Compromised training, corrupted corpus	Multi-agent validation, anomaly pattern review, model consensus
Sensor Spoofing	Fake data, unauthenticated sensors	Multi-modal fusion, spoof detection agent, human review trigger

Table 1. Mapping each attack with their vulnerability and its mitigation

4. Methodology / Block Diagram

This section describes how the system will be realized and tested; each step maps to the logical components represented in Figure 1.



I. Data Input

- **Purpose:** Capture network logs and raw telemetry that describe what the system is doing.
- **Inputs:** Packet captures, NetFlow/IPFIX summaries, API access logs, authentication logs, system metrics, model update events, heartbeat messages.
- **Outputs:** Raw log files or message streams ready for preprocessing.
- **Notes / Example:** Use tools like Wireshark or other network collectors to record traffic. For DoS this will show many requests; for MitM it may show unusual TLS handshakes; for poisoning it will show model update uploads; for spoofing it may show duplicate heartbeats.

II. Data Preprocessing

- **Purpose:** Clean and reshape raw logs; remove unwanted columns and make data uniform.
- **Inputs:** Raw logs and captures from I.
- **Outputs:** Normalized, time-ordered records (CSV, JSON, or messages) with consistent fields.
- **Steps:** parse timestamps, remove irrelevant fields, convert IPs and ports to a standard format, tag missing fields, add basic metadata (device id, zone).
- **Notes / Example:** Preprocessing converts a raw pcap into a list of request records. For DoS you derive per-second request counts; for MitM you extract TLS certificate fields; for poisoning you extract model uploader IDs and hashes.

III. Messaging Layer

- **Purpose:** Insert preprocessed events into a messaging system so agents can read them.
- **Inputs:** Normalized records from II.
- **Outputs:** Messages on Kafka topics or Pub/Sub channels accessible to agents
- **Notes / Example:** Use Kafka or Google Pub/Sub to stream events. Agents subscribe to the topic for real-time processing. This ensures scale and decouples ingestion from processing.

IV. Agentic Pipeline

This is the main boxed pipeline in the diagram. Each agent is a step in the pipeline. We explain each sub-box in order.

- **Data Ingestion Agent**
  - **Purpose:** Read messages from the Messaging Layer and store them for immediate processing.
  - **Inputs:** Messages from III.
  - **Outputs:** Structured event objects forwarded to the Event Classification Agent; short-term buffers for replay.
  - **Details:** Performs basic validation and timestamps; groups related messages into an event (e.g., multiple requests from same IP into one event).
  - **Example:** When many requests arrive, it bundles them as a single "high-request" event for classification.
- **Event Classification Agent**
  - **Purpose:** Decide whether each event looks like normal activity or a suspicious event and label it.
  - **Inputs:** Structured events from Data Ingestion Agent.
  - **Outputs:** Labeled events (e.g., DoS-suspect, MitM-suspect, Poisoning-suspect, Spoofing-suspect, benign) with confidence scores.
  - **Techniques:** Rule-Based Thresholds, Statistical Anomaly Detection, Supervised ML Classifier.
  - **Notes / Example:** For DoS, a rule might flag >1000 requests/min from the same IP. A statistical detector might spot an unusual spike in request entropy. A classifier can combine many features and output a score.
- **RAG Retrieval Agent**
  - **Purpose:** Pull context documents and past incidents from the knowledge base to help explain the event.
  - **Inputs:** Labeled event and its metadata.
  - **Outputs:** Retrieved documents, past incident examples, and protocol references passed to the Reasoning Agent.
  - **Details:** RAG stands for retrieval-augmented generation: retrieve the right playbook or textbook section to explain "what this looks like" and "what others did".
  - **Example:** For a DoS-suspect event, it returns a past incident where blocking a /24 helped; for a MitM-suspect it returns TLS checklist items.

- **Reasoning Agent**
    - **Purpose:** Convert the raw label and retrieved context into a narrative incident-response assessment.
    - **Inputs:** Labeled event plus documents from RAG Retrieval Agent.
    - **Outputs:** Human-readable assessment describing the threat, severity, possible causes, and recommended actions.
    - **Details:** The Reasoning Agent synthesizes: "This pattern matches SYN flood. Probability high. Recommended immediate step: throttle source, notify operator."
    - **Example:** For poisoning-suspect it might say: "Model update from node X deviates from expected label distribution — hold update and run vetting."
- **Planning Agent**
    - **Purpose:** Build an initial plan of actions (task list) to mitigate the threat.
    - **Inputs:** Assessment from Reasoning Agent, system policies, safety priorities.
    - **Outputs:** Concrete steps (e.g., add rate-limit rule, suspend update, rotate cert).
    - **Details:** Plans include rollback options and estimated impact. They are tailored to minimize disruption while addressing risk.
    - **Example:** For DoS, plan = apply rate-limit to endpoint for 5 minutes, start blocking IPs, and open a ticket.
- **Human-in-the-Loop Agent**
    - **Purpose:** Present the plan to an operator and accept approval, modification, or rejection.
    - **Inputs:** Plan from Planning Agent.
    - **Outputs:** Decision (approve, modify, decline) and possibly adjusted plan back to Planner.
    - **Details:** Enables operator oversight for high-risk actions. Also records operator notes.
    - **Example:** Operator approves temporary block but requests to whitelist a service IP.

5. Requirements

- **Software**
  - Kafka or Google Pub/Sub.
  - Wireshark or similar tools which will help capture required logs.
  - ML Frameworks like Pytorch or Tensorflow & Scikit-learn.
  - Agent Orchestration like LangGraph or Google ADK.
  - RAG Components like SentenceTransformers/all-MiniLM for embeddings; FAISS or Pinecone for vector search; a small LLM (local or cloud) for synthesis if available.
  - Python
  - A2A Protocol for connecting the agents.
- **Tools**
  - Docker for automation & deployment.
  - IDE - VSCode.
- **Datasets**
  - We will generate synthetic data and simulate attacks to capture representative logs

6. Proposed Evaluation Measures

- **Precision, Recall, F1-Score** — report per attack class (DoS, MitM, Poisoning, Spoofing).
- **False Alarm Rate (FAR)** — percentage of benign events incorrectly flagged. Aim to minimize to keep operator workload reasonable.
- **False Negative Rate (FNR)** — missed attacks; critical to keep low for safety-related incidents.
- **Detection Latency** — time from attack start to first detection at edge and central. Target: sub-second for extreme volumetric DoS; low seconds for application-layer anomalies.
- **Threat Isolation Rate:** % of successful endpoint containment attempts.
- **Model Integrity:** Accuracy drop under adversarial/poisoned data.
- **Auditability:** % of alerts with explainable RAG-generated reports.

7. Conclusion

This project delivers a modular, AI-driven security layer for smart lighting systems, capable of detecting and mitigating multiple cyber threats using only telemetry and logs. The staged implementation—from ingestion to robust central classification and enforcement—will improve resilience while preserving service availability. The design's reproducibility makes it suitable for academic validation and potential real-world adaptation.

References

[1] N. N. Hussein and A. Nhlabatsi, "Living in the Dark: MQTT-Based Exploitation of IoT Security Vulnerabilities in ZigBee Networks for Smart Lighting Control," Sensors, vol. 23, no. 6, p. 3333, 2023. Available: https://www.mdpi.com/1424-8220/23/6/3333

[2] I. Priyadarshini, "Anomaly Detection of IoT Cyberattacks in Smart Cities Using Federated Learning and Split Learning," Big Data and Cognitive Computing, vol. 8, no. 3, p. 21, 2024. Available: https://www.mdpi.com/2504-2289/8/3/21

[3] H. Zhang, X. Luo, and P. Chen, "Intelligent DDoS Detection for Smart IoT Networks Using Hybrid Deep Learning," Computer Networks, vol. 229, 109744, 2023.

[4] Ö. Sen, D. van der Velde, P. Linnartz, et al., "Investigating Man-in-the-Middle-based False Data Injection in a Smart Grid Laboratory Environment," arXiv preprint arXiv:2110.09162, 2021. Available: https://arxiv.org/abs/2110.09162

[5] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. McDaniel, "Adversarial Perturbations Against Deep Neural Networks for Malware Classification," arXiv preprint arXiv:1606.04435, 2017. Available: https://arxiv.org/abs/1606.04435

[6] F. Wang, X. Wang, X. Ban, et al., "Data Poisoning Attacks in Intelligent Transportation Systems: A Survey," arXiv preprint arXiv:2407.15855, 2024. Available: https://arxiv.org/abs/2407.15855

[7] R. Gupta, A. Singh, and P. Yadav, "Sensor Spoofing Attack Detection in Smart Cities Using Multi-Modal Data Fusion," IEEE Access, vol. 10, pp. 45671–45683, 2022.

[8] Cybersecurity and Infrastructure Security Agency (CISA), "The Future of Smart Cities: Cyber-Physical Infrastructure Risk," 2019. Available: https://www.cisa.gov/sites/default/files/documents/OCIA-%20The%20Future%20of%20Smart%20Cities-%20Cyber-Physical%20Infrastructure%20Risk.pdf

[9] N. Moustafa and J. Slay, "The UNSW-NB15 Dataset: A Comprehensive Network Dataset for Intrusion Detection Systems," UNSW Canberra Research, 2015. Available: https://research.unsw.edu.au/projects/unsw-nb15-dataset

[10] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP," arXiv preprint arXiv:2005.11401, 2020. Available: https://arxiv.org/abs/2005.11401

[11] "Cybersecurity Risk Assessment in Smart City Infrastructures," Sensors (MDPI), 2023. Available: https://www.mdpi.com/2075-1702/9/4/78